



Questions de réflexion sur le Mini-Projet 1 : **PasswordCracker v1**

Membres du groupe 4

Ndeye Maty GAYE
Mamadou Lamarana DOUCOURE
Assatou DRAME KEITA
Souleymane Sirima MBODJ
Serigne Abdoul Aziz NDIAYE

Lien github du projet :

<https://github.com/mamadou-doucoure/PasswordCracker.git>

Q1) Quels avantages apporte la fabrique simple ?

Le principal avantage est la centralisation : toute la logique de création des objets HashCracker est regroupée dans une seule classe dédiée plutôt que dispersée dans le programme.

- Centralisation de la création : `HashCrackerFactory` rassemble tous les `new DictionaryHashCracker()` et `new BruteForceHashCracker()` en un seul endroit, au lieu de les éparpiller dans le code appelant.
- Découplage : `Main` ne manipule que le type abstrait `HashCracker` et ne connaît jamais les classes concrètes ni leurs détails de construction.
- Simplicité d'utilisation : un simple appel `HashCrackerFactory.create("DICO")` suffit pour obtenir la bonne stratégie, sans exposer la logique interne de fabrication.
- Lisibilité et testabilité accrues : le code appelant reste concis, et l'on peut tester la fabrique indépendamment du reste de l'application.

Q2) Quels sont ses inconvénients ?

La simplicité de ce patron a une contrepartie : la fabrique doit connaître, dès sa rédaction, l'ensemble des stratégies possibles.

- Connaissance à l'avance obligatoire : la fabrique contient un `switch` (ou une suite de `if / else`) qui énumère explicitement chaque code, comme `"DICO"` et `"BRUTE"`.
- Violation du principe Open/Closed (voir question 4) : il est impossible d'ajouter une nouvelle stratégie sans modifier le code existant de la fabrique.
- Méthodes statiques : `HashCrackerFactory` expose des méthodes statiques, donc difficiles à sous-classer ou à remplacer par une autre implémentation ; il n'y a pas de polymorphisme possible sur la fabrique elle-même.
- Passage à l'échelle limité : plus le nombre de stratégies augmente, plus la fabrique grossit et devient un point de couplage central, puisqu'elle doit importer et connaître toutes les classes concrètes du projet.

Q3) Que faut-il modifier lorsqu'une nouvelle stratégie est ajoutée ?

Prenons l'exemple de l'ajout d'une nouvelle méthode de cassage, par exemple une attaque par rainbow table identifiée par le code "RAINBOW". Deux modifications sont nécessaires :

1. Créer une nouvelle classe, par exemple `RainbowTableHashCracker`, qui implémente l'interface `HashCracker`.
2. Modifier le code existant de `HashCrackerFactory.create(...)` pour ajouter un nouveau cas dans le `switch` : `case "RAINBOW": return new RainbowTableHashCracker();`

Q4) La fabrique respecte-t-elle le principe Open/Closed ?

Non. Ajouter une stratégie impose de modifier le corps de `HashCrackerFactory.create(...)`, donc de rouvrir une classe déjà écrite et déjà testée.

Or, dans le cas de la Fabrique Simple, ajouter une stratégie (question 3) impose précisément de modifier le corps de la méthode `create(...)` en y ajoutant un nouveau case au switch. La classe existante n'est donc ni fermée à la modification, ni pleinement ouverte à l'extension : le principe Open/Closed n'est pas respecté.

D'ailleurs, la Simple Factory n'est pas à proprement parler un patron de conception officiel du Gang of Four (GoF) : c'est plutôt un idiome pratique, mais limité. C'est justement cette limitation qui sera corrigée dans le prochain mini-projet, probablement à l'aide d'un véritable Factory Method ou d'un système d'enregistrement dynamique des stratégies (registre de fabriques), qui permettront d'ajouter de nouvelles stratégies sans toucher au code existant.

Conclusion

La Fabrique Simple apporte un premier niveau de découplage bienvenu entre le code appelant et les classes concrètes de `HashCracker`, mais elle atteint vite ses limites dès que le nombre de stratégies grandit : elle viole le principe Open/Closed en exigeant la modification d'une classe existante à chaque ajout. Le prochain mini-projet s'attaquera à cette limite en migrant vers un patron de conception plus extensible, tel que le Factory Method ou un registre de stratégies.